13. 검증자 (Verifier) 증명 검증 프로세스 분석

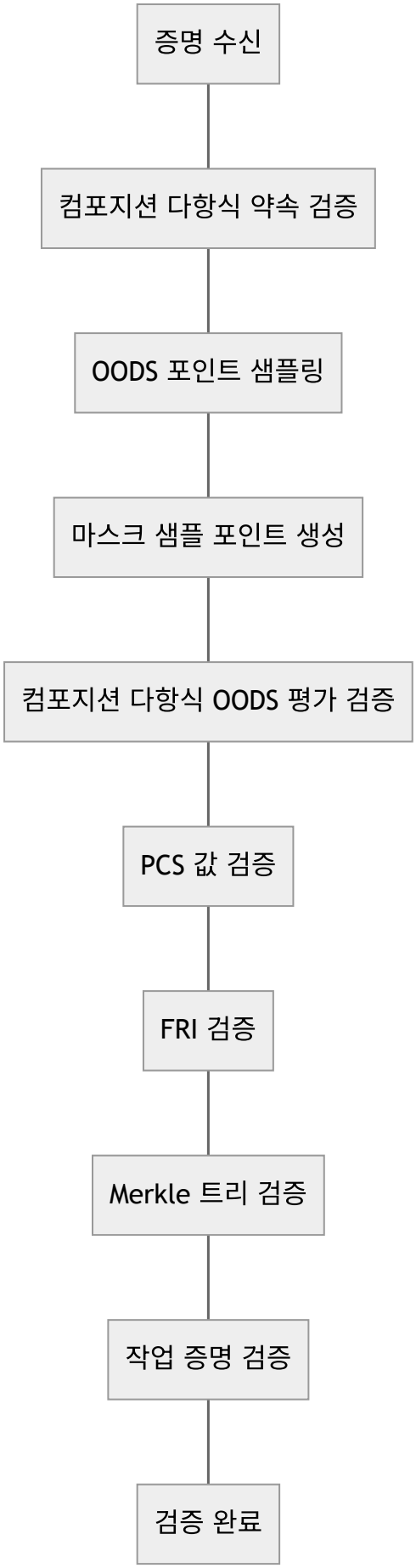
개요

Stwo의 Circle STARK 검증자는 증명자가 생성한 증명을 효율적이고 안전하게 검증하는 핵심 컴포넌트입니다. 검증 프로세스는 여러 단계로 구성되어 있으며, 각 단계에서 암호학적 보안성을 보장하면서도 최적화된 성능을 제공합니다.

1. 검증 프로세스 전체 구조

1.1 주요 검증 단계

Circle STARK 검증 프로세스는 다음과 같은 주요 단계들로 구성됩니다:


증명 수신
컴포지션 다항식 약속 검증
OODS 포인트 샘플링
마스크 샘플 포인트 생성
컴포지션 다항식 OODS 평가 검증
PCS 값 검증
FRI 검증
Merkle 트리 검증
작업 증명 검증
검증 완료

1.2 핵심 검증자 구조

```rust
// 메인 검증 함수 구조
pub fn verify<MC: MerkleChannel>(
components: &[&dyn Component],
channel: &mut MC::C,
commitment_scheme: &mut CommitmentSchemeVerifier<MC>,
proof: StarkProof<MC::H>,
) -> Result<(), VerificationError>
```

2. 단계별 검증 프로세스 분석

2.1 컴포지션 다항식 약속 검증

검증자는 먼저 증명에 포함된 컴포지션 다항식의 약속을 검증합니다:

```rust
// 컴포지션 다항식 약속 읽기
commitment_scheme.commit(
*proof.commitments.last().unwrap(),
&[components.composition_log_degree_bound(); SECURE_EXTENSION_DEGREE],
channel,
);
```

주요 특징:

- 컴포지션 다항식의 차수 경계 확인
- Merkle 루트를 채널에 믹싱하여 Fiat-Shamir 변환 적용

- 보안 확장 차수(SECURE_EXTENSION_DEGREE)만큼 반복

2.2 OODS (Out-Of-Domain Sampling) 포인트 생성

```
// OODS 포인트 무작위 샘플링
let oods_point = CirclePoint::<SecureField>::get_random_point(channel);
```

OODS 기법의 핵심:

- 도메인 외부의 무작위 포인트에서 다항식 평가
- 증명자가 미리 예측할 수 없는 포인트 선택
- Circle 기하학을 활용한 효율적인 포인트 생성

2.3 마스크 샘플 포인트 계산

```
// 마스크 샘플 포인트 생성
let mut sample_points = components.mask_points(oods_point);
// 컴포지션 다항식 마스크 포인트 추가
sample_points.push(vec![vec![oods_point]; SECURE_EXTENSION_DEGREE]);
```

마스크 포인트의 역할:

- 각 컴포넌트의 제약 조건 평가에 필요한 포인트들
- OODS 포인트를 기준으로 상대적 위치 계산
- 트레이스 테이블의 행 간 관계 검증을 위한 포인트 집합

2.4 컴포지션 다항식 OODS 평가 검증

```
// 컴포지션 다항식 OODS 평가 추출
let composition_oods_eval = proof.extract_composition_oods_eval()
.ok_or(VerificationError::InvalidStructure(...))?;


// OODS 평가 일치성 검증
if composition_oods_eval != components.eval_composition_polynomial_at_point(
oods_point,
&proof.sampled_values,
random_coeff,
) {
return Err(VerificationError::OodsNotMatching);
}
```

DEEP-ALI 검증:

- 컴포지션 다항식의 OODS 평가와 트레이스 OODS 값들의 일치성 확인
- 무작위 계수를 사용한 선형 결합으로 보안성 강화
- 증명자가 올바른 제약 조건을 만족하는지 검증

3. FRI (Fast Reed-Solomon Interactive Oracle Proof) 검증

3.1 FRI 검증자 구조

```
pub struct FriVerifier<MC: MerkleChannel> {
config: FriConfig,
```

```
first_layer: FriFirstLayerVerifier<MC::H>,
inner_layers: Vec<FriInnerLayerVerifier<MC::H>>,
last_layer_domain: LineDomain,
last_layer_poly: LinePoly,
queries: Option<Queries>,
}
```

3.2 FRI 약속 단계 검증

```
pub fn commit(
channel: &mut MC::C,
config: FriConfig,
proof: FriProof<MC::H>,
column_bounds: Vec<CirclePolyDegreeBound>,
) -> Result<Self, FriVerificationError>
```

약속 단계 검증 과정:

1. **첫 번째 레이어 검증**: Circle 다항식들의 약속 검증
2. **내부 레이어 검증**: 각 폴딩 단계의 약속 검증
3. **마지막 레이어 검증**: 최종 다항식의 차수 검증

3.3 FRI 디커밋 단계 검증

```
pub fn decommit(
```

```
mut self,
first_layer_query_evals: ColumnVec<Vec<SecureField>>,
) -> Result<(), FriVerificationError>
```

디커밋 검증 과정:

1. **쿼리 위치 샘플링**: 무작위 쿼리 위치 생성
2. **첫 번째 레이어 검증**: Circle 다항식 평가값 검증
3. **내부 레이어 폴딩 검증**: 각 레이어의 폴딩 일치성 검증
4. **마지막 레이어 검증**: 최종 다항식 평가 검증

4. Merkle 트리 검증

4.1 Merkle 검증자 구조

```
pub struct MerkleVerifier<H: MerkleHasher> {
pub root: H::Hash,
pub column_log_sizes: Vec<u32>,
pub n_columns_per_log_size: BTreeMap<u32, usize>,
}
```

4.2 Merkle 디커밋 검증

```
pub fn verify(
&self,
```

```
queries_per_log_size: &BTreeMap<u32, Vec<usize>>,
queried_values: Vec<BaseField>,
decommitment: MerkleDecommitment<H>,
) -> Result<(), MerkleVerificationError>
```

Merkle 검증 과정:

1. **해시 증인 검증**: 필요한 해시 값들의 유효성 확인
2. **컬럼 증인 검증**: 쿼리되지 않은 컬럼 값들의 유효성 확인
3. **루트 재계산**: 제공된 값들로부터 Merkle 루트 재계산
4. **루트 일치성 검증**: 계산된 루트와 약속된 루트의 일치성 확인

5. 배치 검증 최적화

5.1 배치 검증 전략

Stwo는 여러 증명을 동시에 검증할 때 효율성을 높이기 위한 배치 검증 기법을 사용합니다:

```
// 배치 검증을 위한 쿼리 위치 통합
let  query_positions_per_log_size = fri_verifier.sample_query_positions(channel)


// 여러 트리에 대한 동시 검증
self.trees
.as_ref()
.zip_eq(proof.decommitments)
.zip_eq(proof.queried_values.clone())
```

```
.map(|((tree, decommitment), queried_values)| {
tree.verify(&query_positions_per_log_size, queried_values, decommitment)
})
```

배치 검증의 장점:

- 공통 쿼리 위치 재사용으로 계산량 감소
- 메모리 접근 패턴 최적화
- 병렬 처리 가능한 구조

5.2 메모리 효율성 최적화

```
// 스트리밍 방식의 증인 소비
let mut hash_witness = decommitment.hash_witness.into_iter();
let mut column_witness = decommitment.column_witness.into_iter();

// 필요한 만큼만 메모리에 로드
let node_values = node_values_iter.take(n_columns_in_layer).collect_vec();
```

6. 보안성 보장 메커니즘

6.1 Fiat-Shamir 변환

```
// 채널을 통한 무작위성 생성
let random_coeff = channel.draw_secure_felt();
```

```
let  oods_point = CirclePoint::<SecureField>::get_random_point(channel);
```

보안성 특징:

- 증명자가 예측할 수 없는 무작위 값 생성
- 이전 단계의 결과를 기반으로 한 결정론적 무작위성
- 재현 가능한 검증 과정

6.2 작업 증명 (Proof of Work) 검증

```
// 작업 증명 검증
channel.mix_u64(proof.proof_of_work);
if  channel.trailing_zeros() < self.config.pow_bits {
return Err(VerificationError::ProofOfWork);
}
```

작업 증명의 역할:

- 증명 생성 비용 증가로 스팸 방지
- 추가적인 보안 계층 제공
- 설정 가능한 난이도 조절

7. 에러 처리 및 견고성

7.1 검증 에러 타입

```
#[derive(Clone, Debug, Error)]
```

```rust
pub enum VerificationError {
#[error("Proof has invalid structure: {0}.")]
InvalidStructure(String),
#[error(transparent)]
Merkle(#[from] MerkleVerificationError),
#[error("The composition polynomial OODS value does not match...")]
OodsNotMatching,
#[error(transparent)]
Fri(#[from] FriVerificationError),
#[error("Proof of work verification failed.")]
ProofOfWork,
}
```

7.2 견고한 에러 처리

```rust
// 구조적 유효성 검사
let composition_oods_eval = proof.extract_composition_oods_eval()
.ok_or(VerificationError::InvalidStructure(...))?;

// 증인 소비 완전성 검사
if hash_witness.next().is_some() {
return Err(MerkleVerificationError::WitnessTooLong);
```

```
}
```

8. 성능 최적화 기법

8.1 병렬 처리

```
// 병렬 검증 가능한 구조
self.trees
.as_ref()
.zip_eq(proof.decommitments)
.map(|((tree, decommitment), queried_values)| {
tree.verify(&query_positions_per_log_size, queried_values, decommitment)
})
```

8.2 메모리 접근 최적화

- 순차적 메모리 접근 패턴 사용
- 캐시 친화적인 데이터 구조
- 불필요한 메모리 할당 최소화

9. 검증자 효율성 분석

9.1 계산 복잡도

- **FRI 검증**: O(log n) 시간 복잡도
- **Merkle 검증**: O(k log n) (k는 쿼리 수)

- **OODS 검증**: O(1) 시간 복잡도

9.2 통신 복잡도

- **증명 크기**: O(log n) 크기
- **쿼리 응답**: O(k) 크기
- **전체 통신량**: 증명자 대비 매우 작은 크기

결론

Stwo의 Circle STARK 검증자는 다층적인 보안 메커니즘과 효율적인 최적화 기법을 통해 강력한 보안성과 우수한 성능을 동시에 제공합니다. OODS 기법, FRI 프로토콜, Merkle 트리 검증, 그리고 작업 증명이 유기적으로 결합되어 완전하고 안전한 검증 시스템을 구성합니다.